Audion — built-in functions

groups numbered by importance · [ ] = optional · / = alternatives · … = same prefix

1 Sound — synths & nodes

define name(params) { ... } builds a SynthDef from UGens (see UGens). Default params: freq 440 · amp 0.1 · pan 0 · gate 1 · out 0.

- **synth(name, k: v, ...)** — start a synth, named args = its params → node id
- **set(node, k: v, ...)** — change params of a running synth; gate: 0 = release
- **free(node)** — hard stop a synth

2 Time & clock

Under lookahead, wait only moves the thread's beat cursor. Threads reacting to input (UI, MIDI in) use sleep_ms.

- **bpm() / bpm(n)** — get / set tempo (set returns old bpm)
- **wait(beats)** — wait N beats
- **wait_ms(ms)** — wait N milliseconds
- **sleep_ms(ms)** — really sleep, beat cursor untouched
- **latency() / latency(ms)** — get / set lookahead (default 500 ms)
- **time()** — seconds since program start
- **beat_now()** — real clock, in beats
- **beat_cursor()** — this thread's beat cursor
- **beat_cursor_set(b)** — move the cursor (after a pause, no catch-up)

3 Pitch & random

*rand(a, b) with whole-number bounds returns a whole number — use rand() * x for floats.*

- **mtof(note [, tuning])** — MIDI → Hz; tuning = .scl path or ratio array
- **ftom(freq [, tuning])** — Hz → MIDI note
- **rand() / rand(min, max)** — random 0..1 / in range
- **seed(v)** — per-thread seed (number or string); seed(false) = off
- **array_rand(arr)** — random element (respects seed)

4 Arrays

Arrays are ordered maps. Assigning or returning an array copies it.

- **count(x)** — length of array or string
- **push(arr, v)** — append → new length (alias array_push)
- **pop(arr)** — remove + return last (alias array_pop)
- **keys(arr)** — all keys as array
- **has_key(arr, k)** — key exists?
- **remove(arr, k)** — remove by key → value
- **in_array(needle, arr)** — value exists?
- **array_shuffle(arr)** — random order
- **array_cycle(arr, n)** — rotate by n (alias array_rotate)
- **array_chunk(arr, size)** — split into chunks
- **array_next(arr [, loop])** — cursor forward → value; array_prev back
- **array_beginning / array_end(arr)** — cursor to first / last
- **array_current(arr) / array_key(arr)** — value / key at cursor

5 Rhythm generators — array_seq_*

Most return arrays of 0/1 steps, intervals, or lists of arrays.

- **array_seq_euclidean(pulses, steps)** — euclidean rhythm [1,0,1,...]
- **array_seq_markov(matrix, start, count)** — states from a transition matrix
- **array_seq_binary_to_intervals("1001..")** — steps → gaps [3, ...]
- **array_seq_intervals_to_binary([2,3,..])** — gaps → "10100.."
- **array_seq_random_correlated(m, s, c, n)** — n randoms 0..m, c = 0 all same, c = m free
- **array_seq_necklaces(n)** — all n-step rotation-unique patterns (n ≤ 30)
- **…_necklaces_m_ones(n, m)** — with exactly m hits
- **…_necklaces_allowed(n, allowed)** — only allowed gap sizes
- **…_necklaces_allowed_m_ones(n, m, allowed)** — both
- **array_seq_compositions(n)** — all ordered splits of n
- **…_compositions_m_parts(n, m) / _allowed(n, a)** — fixed count / allowed parts
- **…_compositions_allowed_m_parts(n, m, a)** — both
- **array_seq_composition_random(n)** — one random split; _m_parts(n, m)
- **array_seq_partitions(n)** — unordered splits; _m_parts / _allowed / _allowed_m_parts
- **array_seq_permutations(arr)** — all orderings
- **array_seq_debruijn(n)** — binary de Bruijn (n ≤ 20)
- **array_seq_christoffel("upper"|"lower", p, q [, n])** — Christoffel word
- **array_seq_cf_convergent(terms) / _cf_sqrt(n)** — continued fractions
- **array_seq_paper_folding(n, m, f)** — paper-folding sequence

6 Melody generators — array_mel_*

- **array_mel_random_walk(start, min, max, step, len)** — bounded random walk
- **array_mel_invert(mel, pivot)** — mirror around pivot
- **array_mel_reverse(mel)** — retrograde
- **array_mel_subset_sample(arr, size)** — random subset
- **array_mel_string_to_indices(str, num_notes)** — text → note indices
- **array_mel_debruijn_k(k, n, v)** — k-symbol de Bruijn, v = variant
- **array_mel_lattice_walk_square(nc, nr, x, y, a, b, n)** — grid walk (x,y) → (a,b), n steps
- **…_lattice_walk_tri / _square_no_retrace (same args)** — triangle grid / no revisits
- **…_square_with_stops(nc, nr, x, y, a, b, stops, n)** — walk with repeats
- **array_mel_lattice_to_melody(grid, path, x, y)** — path string → notes
- **array_mel_automaton(auto, start, ends, len)** — finite-state melody
- **array_mel_probabilistic_automaton(auto, start, len)** — random-transition version

7 Probability & ML — ml_*

- **ml_weighted_choice(values, weights)** — pick one by weight
- **ml_markov_train(notes, order)** → model
- **ml_markov_generate(model, len [, start])** → notes
- **ml_markov_next(model, context)** → [[value, prob], ...]
- **ml_softmax(values [, temp])** → probabilities
- **ml_normalize(values) / ml_entropy(probs)** — sum to 1 / bits
- **ml_kalman_filter(values, Q, R)** — smooth a noisy array
- **ml_kalman_smooth(start, end, steps [, vol])** — organic ramp start → end
- **ml_kalman_state(x, Q, R)** — live filter state
- **ml_kalman_update(state, x) / _predict(state, n)** — feed one value / extrapolate

8 Buffers & recording

- **buffer_load(path)** — load sound file → buffer id
- **buffer_load_stereo(path)** — always 2 channels
- **buffer_alloc(frames [, ch])** — empty buffer
- **buffer_read(id, path)** — load file into existing buffer
- **buffer_set(id, [v, ...])** — write values from frame 0
- **buffer_free(id)** — free buffer
- **buffer_query(id)** → [frames, channels, samplerate]
- **buffer_peaks(id, n)** — n peaks for waveform drawing
- **buffer_beats(id)** → onsets, beats, bpm, confidence
- **buffer_stream_open(path) / _close(id)** — disk streaming (use stream_disk)
- **record_start([path])** — record master out → path
- **record_stop() / record_path()** — stop / current file

9 UGens — inside define { } only

*Signals take math (+ – * /). Args are positional; missing ones use defaults.*

Oscillators

sine(f) · saw(f) · square(f) / pulse(f, w) · tri(f) · blip(f, harms) · var_saw(f, w) · sync_saw(sync, f) · fsin_osc(f) · lf_par(f) · lf_cub(f) · pm_osc(car, mod, index, phase)

Noise & triggers

noise() / white() · pink() · brown() · gray() · clip_noise() · crackle(chaos) · dust2(d) · Dust(d) · Impulse(f)

Filters

lpf(sig, cut) · hpf(sig, cut) · bpf(sig, f, rq) · rlpf(sig, cut, rq) · rhpf(sig, cut, rq) · brf(sig, f, rq) · resonz(sig, f, bwr) · moog_ff(sig, f, gain) · ringz(sig, f, decay) · formlet(sig, f, atk, dec) · mid_eq(sig, f, rq, db) · one_pole(sig, coef) · two_pole(sig, f, radius) · leak_dc(sig) · lpz1 / lpz2 / hpz1 / hpz2(sig)

Envelopes

env(gate, atk, sus, rel) — frees synth when done, sus 0 = perc · adsr(gate, atk, dec, sus, rel) — retrigger, never frees · env_timed(atk, dec, sus, hold, rel, done) · env_perc(gate, atk, rel, curve) · linen(gate, atk, sus, rel) · line(a, b, dur) · xline(a, b, dur) · decay(sig, t) · decay2(sig, atk, dec)

LFOs (control rate)

lfo_sine(f) · lfo_saw(f) · lfo_tri(f) · lfo_pulse(f, w) · lfo_noise(f) smooth · lfo_noise2(f) · lfo_step(f) stepped · vibrato(f, rate, depth, delay, onset, rvar, dvar)

Smoothing

lag(sig, t) · lag2(sig, t) · ramp(sig, t) · slew(sig, up, down)

Delay & reverb

delay(sig, time, decay) · delay_n / _l / _c(sig, max, time) · comb_n / comb_c(sig, max, time, decay) · allpass_n / _l / _c(sig, max, time, decay) · delay1 / delay2(sig) · pluck(sig, trig, max, time, decay, coef) · reverb(sig, mix, room, damp) · freeverb2(l, r, mix, room, damp) · gverb(sig, room, time, damp, bw, spread, dry, early, tail, maxroom) · gverb_stereo(…)

Drive & dynamics

dist(sig, amt) · tanh(sig) · atan(sig) · softclip(sig) · wrap(sig, lo, hi) · fold(sig, lo, hi) · compander(sig, ctrl, thresh, below, above, clamp, relax) · limiter(sig, level, dur) · normalizer(sig, level, dur)

Pitch

pitch_shift(sig, win, ratio, pdisp, tdisp) · freq_shift(sig, f, phase) · pitch(sig, init, min, max) tracker

Samples & buffers

`sample("f.wav", root:, loop:, start:, …)` · `PlayBuf(ch, buf, rate, trig, start, loop)` · `buf_rd(ch, buf, phase)` · `phasor(trig, rate, start, end [, reset])` · `buf_wr(sig, buf, phase, loop)` · `record_buf(sig, buf, offset, rec, pre, run, loop)` · `local_buf(frames, ch)` · `buf_rate_scale(buf)` · `curve_k(buf, hz, once, size)` · `stream_disk(buf)` · `stream_disk_variable_rate(buf, rate)`

Granular

`warp(ch, buf, ptr, pitch, window, overlaps, rand)` · `GrainBuf(ch, trig, dur, buf, rate, pos, interp, pan, env, max)` · `GrainSin(ch, trig, dur, f, pan, env, max)` · `GrainFM(ch, trig, dur, car, mod, index, pan, env, max)` · `grains_t(ch, trig, buf, rate, center, dur, pan, amp, interp)`

Triggers & analysis

`changed(x)` · `latch(sig, trig)` · `gate(sig, trig)` · `coin_gate(prob, trig)` · `pulse_count(trig, reset)` · `sweep(trig, rate)` · `TRand(lo, hi, trig)` · `t_irand(lo, hi, trig)` · `t_exprand(lo, hi, trig)` · `amplitude(sig, atk, rel)` · `peak(sig, trig)` · `running_sum / _max / _min(sig, n)` · `median(len, sig)` · `zero_crossing(sig)` · `send_reply(hz, addr, values)` → back to Audion

Routing

`out(bus, sig)` · `pan(pos, sig)` · `pan4(sig, x, y, lvl)` · `balance2(l, r, pos, lvl)` · `splay(sigs, spread, lvl, center)` · `in(bus, ch)` · `local_in(ch) / local_out(sig)` — feedback, 1 block late

Utility

`array(a, b, …)` · `array_get(arr, i)` · `Clip(sig, lo, hi)` · `Wrap(sig, lo, hi)` · `LinLin(sig, inlo, inhi, outlo, outhi)` · `LinExp(…)` · any other name = raw SC UGen

10 MIDI

- **`midi_config()`** — list output ports
- **`midi_config(name | index)`** — connect output
- **`midi_note(note, vel [, ch])`** — note on; vel 0 = off
- **`midi_cc(cc, val [, ch])`** — control change
- **`midi_program(p [, ch])`** — program change
- **`midi_listen(port, fn(event))`** — input; blocks → run in a thread
- **`midi_bpm_sync(port, on)`** — follow incoming MIDI clock
- **`midi_clock() / midi_start() / midi_stop()`** — send clock tick / start / stop
- **`midi_panic()`** — all notes off, 16 channels
- **`midi_out(b1, b2 [, b3])`** — raw message
- **`midi_read(path) / midi_write(path, events)`** — .mid file → events / events → file

11 OSC & Ableton Link

- **`osc_config(["host:port"])`** — get / set send target
- **`osc_send(path, ...args)`** — send message
- **`osc_listen(port)`** — start listening
- **`osc_recv()`** — next message or nil (non-blocking)
- **`osc_close(["sender" | "all"])`** — close sockets
- **`link_enable([on]) / link_disable()`** — join / leave Link session
- **`link_is_enabled() / link_peers()`** — state / peer count
- **`link_beat() / link_phase()`** — session beat / phase
- **`link_quantum([n])`** — get / set bar length
- **`link_play() / link_stop() / link_is_playing()`** — transport
- **`link_request_beat(b)`** — jump session beat

12 UI (run with --ui)

Widget ids are strings. has_changed() clears on read.

- **`ui_desktop()`** — new window → ui
- **`ui.window(title, w, h) / ui.theme(key, ...)`** — size / window-wide style
- **`ui.widgets.slider(id) / slider_v / slider_range`** — 0..1 by default
- **`ui.widgets.knob / number / button / toggle(id)`**
- **`ui.widgets.dropdown(id, "A", "B", ...)`** → index
- **`ui.widgets.text_label(id, txt) / text_input(id)`**
- **`ui.widgets.array(id, n) / array_numbers(id, n)`** — toggle / number rows
- **`ui.widgets.piano(id, octaves, start)`** — .keyboard(on) .hold(on) .flash(note [, ms])
- **`ui.widgets.canvas(id, w, h)`** — 2D drawing surface
- **`w.value() / w.has_changed() / w.set(v)`** — read / changed? / write
- **`w.min(n) .max(n) .label(s) .style(key, ...)`** — configure
- **`c.clear() .fill(r,g,b) .rect / .rect_outline(x,y,w,h,r,g,b)`** — canvas draw
- **`c.circle / .circle_outline(cx,cy,rad,r,g,b)`**
- **`c.line(x1,y1,x2,y2,r,g,b [,w]) .text(x,y,s,size,r,g,b)`**
- **`c.mouse_x() .mouse_y() .mouse_down() .events()`** — mouse + keys
- **`c.pick_file(exts...) / c.picked()`** — file dialog, poll for path

13 Strings & types

- **`str(v) / int(v) / float(v) / bool(v)`** — convert
- **`hex(n [, width]) / bin / oct`** — number → string
- **`str_explode(delim, s) / str_join(glue, arr)`** — split / join
- **`str_replace(needle, repl, s)`** — replace all
- **`str_contains(needle, s)`** — contains?
- **`str_substr(s, start [, len])`** — substring
- **`str_length(s) / str_trim(s)`**
- **`str_upper(s) / str_lower(s)`**
- **`str_starts_with(pre, s) / str_ends_with(suf, s)`**
- **`json_encode(v) / json_decode(s)`** — JSON

14 Math — math_*

- **`math_map(v, inlo, inhi, outlo, outhi)`** — remap range
- **`math_clamp(n, lo, hi) / math_wrap / math_fold`** — keep in range
- **`math_lerp(a, b, t)`** — blend
- **`math_round(n [, places]) / floor / ceil / trunc`**
- **`math_fract / abs / sign(n)`**
- **`math_min(...) / math_max(...)`** — any number of args
- **`math_pow(b, e) / sqrt / cbrt / exp / expm1`**
- **`math_log(n [, base]) / log2 / log10 / log1p`**
- **`math_fmod(x, y) / math_intdiv(a, b) / hypot(x, y)`**
- **`math_sin cos tan asin acos atan atan2(y, x)`** — radians
- **`math_sinh cosh tanh asinh acosh atanh`**
- **`math_deg2rad / rad2deg · math_pi() / math_e()`**
- **`math_is_nan / is_finite / is_infinite`**

15 Files & data

Relative paths resolve from the script's folder. Failures return false.

- **`file_read(path [, off, len])`** — text
- **`file_write(path, s) / file_append(path, s)`**
- **`file_exists / file_delete / file_size(path)`**
- **`file_open(path, "r"|"w"|"a")` → `h`** — handle based IO:
- **`file_line(h) / file_read_chunk(h, n)`**
- **`file_write_handle(h, s) / file_seek(h, off) / file_tell(h)`**
- **`file_close(h)`**
- **`dir_scan / dir_exists / dir_create / dir_delete(path)`**
- **`file_read_bytes(path [, off, len]) / file_write_bytes`** — binary
- **`bytes_len / bytes_get(b, i) / bytes_slice(b, s [, n])`**
- **`bytes_to_array(b) / array_to_bytes(arr)`**
- **`sqlite_open(path | ":memory:")` → `h`** — SQLite:
- **`sqlite_exec(h, sql, ...params)`** → rows changed
- **`sqlite_query(h, sql, ...params)`** → rows
- **`sqlite_tables(h) / sqlite_table_exists(h, name)`**
- **`sqlite_close(h)`**

16 Network & DMX

- **`net_http(method, url [, body, headers])`** → response
- **`net_connect(host, port) / net_listen(host, port)`** — TCP → handle
- **`net_accept(h) / net_read(h [, max]) / net_write(h, s)`**
- **`net_udp_bind([host,] port)`** — UDP → handle
- **`net_udp_send(h, host, port, s) / net_udp_recv(h [, max])`**
- **`net_close(h)`** — TCP or UDP
- **`dmx_connect(host [, port]) / dmx_universe(n)`** — Art-Net out (port 6454)
- **`dmx_set(ch, val) / dmx_set_range(start, vals)`**
- **`dmx_send() / dmx_blackout()`**

17 System

- **`print(...)`** — print values, space separated
- **`assert(cond [, msg])`** — error if false
- **`eval(code)`** — run Audion code string
- **`exec(cmd, args...)`** → [stdout, stderr, status] or false
- **`hash("md5"|"sha256"|"sha512", s)`** — hex digest
- **`date(fmt [, ts])`** — PHP style: "Y-m-d H:i:s"
- **`timestamp() / timestamp_ms()`** — unix time
- **`console_read() / console_read_key()`** — stdin line / key
- **`console_read_password([prompt]) / console_error(s)`**
- **`os_args() / os_exit([code])`**
- **`os_env_get(k) / os_env_set(k, v) / os_env_list()`**
- **`os_cwd() / os_chdir(p) / os_script_dir()`**
- **`os_name / os_hostname / os_username / os_home()`**
- **`os_pid() / os_ppid()`** — long names: os_process_id …